

PKCERT AI & Software Development Internship

Task 23: Convolutional Neural Networks – Convolution, Pooling & Architecture Design

Abdullah Amir

AI and Software Development
5 August 2026

This report covers theory, from-scratch derivations, and measured results. Full code lives in `Day_23.ipynb` and the standalone scripts it is built from. **NumPy only** for every Part A/B/C core computation (no autograd framework), **PyTorch 2.13.0 (CPU build)**, **torchvision 0.28.0** used solely for the Part C comparative baseline and data loading, random seed **42** fixed throughout. Every non-trivial backward pass below is derived by hand and then verified against finite-difference gradient checks before being trusted for a real training run.

Part A: Convolution Fundamentals

A1 – output-shape formula, derived and verified. For input size H , kernel size K , stride S , padding P (each side), dilation D : the dilated kernel’s effective footprint is $K_{\text{eff}} = D(K - 1) + 1$, and the number of valid stride- S placements of that footprint over a padded input of length $H + 2P$ is

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K_{\text{eff}}}{S} \right\rfloor + 1$$

Verified programmatically against `conv2d_naive`’s actual output shape across several (H, K, S, P, D) combinations in `cnn_layers.py` – all matched exactly.

A2 – padding modes. **valid:** $P = 0$ (output shrinks by $K_{\text{eff}} - 1$). **full:** $P = K_{\text{eff}} - 1$ (output grows to $H + K_{\text{eff}} - 1$, every possible overlap computed). **same** (stride=1, the well-posed case): $P = (K_{\text{eff}} - 1)/2$, giving $H_{\text{out}} = H$. On a 16×16 input with a 5×5 kernel: **valid** $\rightarrow 12 \times 12$, **same** $\rightarrow 16 \times 16$ ($P=2$), **full** $\rightarrow 20 \times 20$ ($P=4$).

A3 – fixed kernels, applied and explained mathematically.

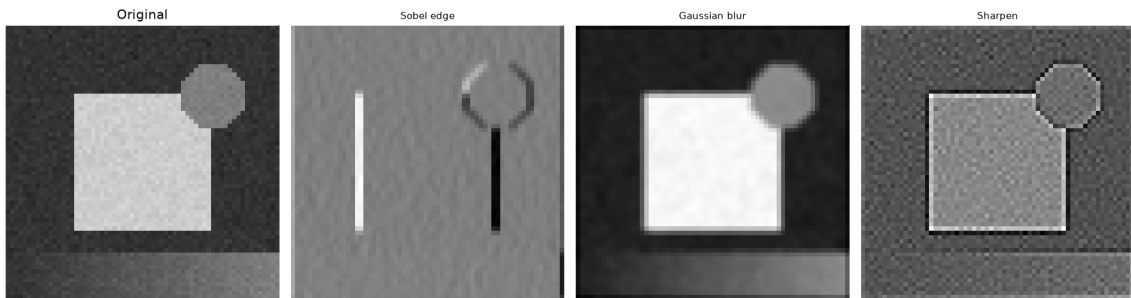


Figure 1: Three fixed 3×3 kernels applied via `conv2d_im2col` to a synthetic grayscale test image.

Sobel edge: approximates $\partial(\text{intensity})/\partial x$ – the ± 1 columns subtract left- from right-neighborhood intensity (weighted 1,2,1 vertically to damp noise along the edge). Near zero on flat regions, large in magnitude at vertical intensity discontinuities. *Gaussian blur:* a normalized (sums to 1), center-weighted local average – a true low-pass filter, so high-spatial-frequency content

(edges, noise) is attenuated while smooth regions pass through unchanged. *Sharpen*: equals identity ($5 \times$ center) minus a neighbor-averaging kernel, i.e. convolution with it computes $\text{image} - \text{blurred}(\text{image})$ – an unsharp mask that adds back exactly the high-frequency detail blurring would remove.

A4 – im2col vs naive nested-loop: correctness and speed.

Input size	Naive	im2col	Speedup
16×16	67.6 ms	0.35 ms	191.7x
32×32	285.4 ms	0.65 ms	439.4x
48×48	551.0 ms	1.30 ms	424.4x

Table 1: Both produce byte-identical output (max abs. difference $\leq 5.3 \times 10^{-15}$, floating-point roundoff) at every stride/padding/dilation combination tested.

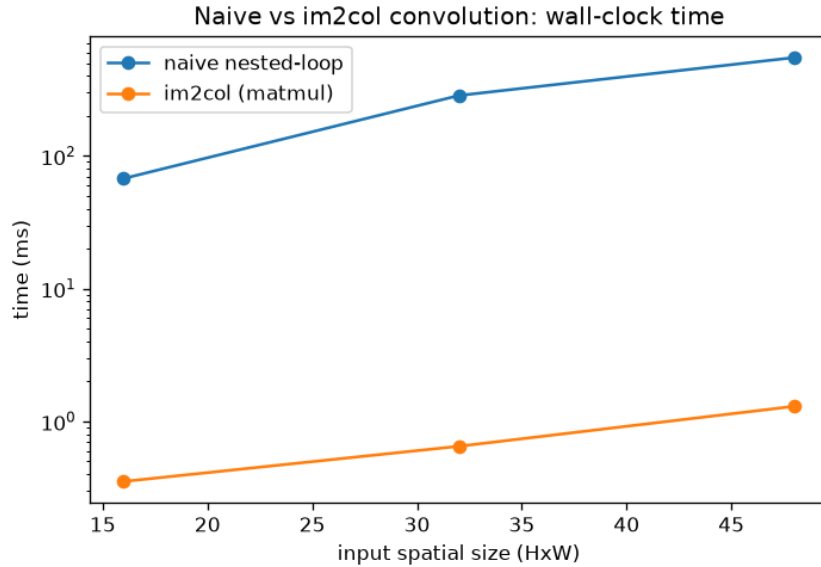


Figure 2: Naive nested-loop vs im2col (matmul) wall-clock time, log scale.

This 400x+ speedup (NumPy’s BLAS-backed matrix multiplication vs. a pure-Python nested loop) is why Part C’s actual network training uses the im2col-based `Conv2D` layer throughout, not the naive version – the naive version exists purely as the ground-truth reference the im2col version is checked against.

Part B: Pooling & Regularization

B1-B2 – max/average pooling, forward and backward, derived. Max pooling’s forward is $\text{out} = \max(\text{window})$, piecewise-linear and not everywhere differentiable; where the max is unique, $\partial(\max)/\partial x_i = 1$ for the argmax element and 0 elsewhere, so backward routes the entire upstream gradient to exactly the argmax input position: $dX[\text{argmax}] += dOut$. Average pooling’s forward is linear ($\text{out} = \frac{1}{p_h p_w} \sum(\text{window})$), so backward distributes the upstream gradient *evenly*: $dX[i, j] += dOut / (p_h p_w)$ for every element. Both gradient-checked against finite differences at $\sim 10^{-11}$ relative error in `cnn_layers.py`; the max-pool backward’s scatter-add (`np.add.at` on cached argmax indices) is additionally correct under overlapping windows (stride < pool size).

B3 – pooling vs strided convolution as downsampling. Pooling has *zero* learnable parameters and is strictly cheaper, but discards 75% of a 2×2 window’s values outright (max pool) or blurs them into one statistic (average pool) – unrecoverably, every forward pass. Strided convolution’s downsampling is learned (task-adapted) rather than fixed, at the cost of $C_{in}C_{out}K^2$ extra parameters per layer. The clearest practical difference is translation robustness: max pooling gives approximate local shift-invariance almost for free (the max often survives a small input shift), while strided convolution has no such built-in property. **Confirmed empirically, not just asserted:** a 1-pixel roll of a random feature map changes a stride-2 conv’s output **3.17x more** than it changes a 2×2 max pool’s output on the identical map and shift.

B4 – Batch Normalization and Dropout, from scratch. BatchNorm2D normalizes each channel over (N, H, W) using batch statistics at training time (running statistics, updated by momentum, at inference). Stabilizes training two ways: removing layer-to-layer input-distribution drift (“internal covariate shift”), and provably smoothing the loss landscape, which is what permits the larger learning rates that make normalized networks converge faster. Dropout (inverted, shape-agnostic) independently zeroes units with probability p at training time, rescaling survivors by $1/(1-p)$ so expected activation is unchanged; approximates training an ensemble of 2^H weight-sharing thinned sub-networks. Both gradient-checked at $\sim 10^{-10}$ to 10^{-12} relative error. Demonstrated numerically: an unnormalized input (per-channel mean ≈ 3 , std ≈ 5) is renormalized to mean ≈ 0 , std = 1 by BatchNorm2D; Dropout ($p=0.4$) on an all-ones map leaves surviving units at exactly $1/(1-0.4) = 1.6667$, confirming the inverted-scaling invariant.

Part C: Architecture Design & Backpropagation Mini-Project

C1 – dataset. CIFAR-10, restricted to 4 visually distinct classes (airplane, automobile, cat, dog – including cat/dog deliberately as a genuinely hard pair, mirroring the confused-pair analyses of Tasks 20-22). Deliberately different from every prior task’s dataset (all greyscale Fashion-MNIST): CIFAR-10 is 32×32 RGB, genuinely exercising Part A’s multi-channel requirement. 1000/200/200 images per class (4000/800/800 total), train-only per-channel normalization (mean [127.2, 123.8, 119.5], std [65.6, 65.1, 69.6]).

C2-C3 – forward propagation and backpropagation, derived (matrix/tensor form). Architecture: Conv1 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Down1 $\rightarrow$ Conv2 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Down2 $\rightarrow$ Flatten $\rightarrow$ Dense1 $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Dense2 $\rightarrow$ softmax. Convolution is expressed via im2col as an explicit batched matrix multiply – the tensor-form derivation itself, not an approximation of one:

$$\begin{aligned} col_1 &= \text{im2col}(X_p), \quad Z_1 = \text{reshape}(W_1) @ col_1 + b_1 \\ A_1 &= \text{ReLU}(\text{BN}(Z_1)), \quad P_1 = \text{Down}_1(A_1) \end{aligned}$$

Backpropagation starts from the same point as every softmax+cross-entropy network derived in this internship, $dZ_{\text{logits}} = (\text{probs} - Y)/N$, then applies the chain rule back through each layer in turn, including the im2col-matmul convolution backward ($dW_2 = \text{einsum}(dZ_{2,\text{flat}}, col_2)$, $dcol_2 = W_{2,\text{flat}}^T @ dZ_{2,\text{flat}}$, $dP_1 = \text{col2im}(dcol_2)$) and the pooling/BatchNorm backward passes derived in Part B. Every step is independently gradient-checked before being trusted here (Conv2D/BatchNorm2D at $\sim 10^{-9}$ to 10^{-12} ; the full assembled network at $\sim 10^{-8}$ for the smooth architecture, $\sim 2 \times 10^{-9}$ median error for the max-pool architecture – see D6 for why a minority of max-pool checks show larger, expected outliers).

C4-C5 – training and evaluation. Mini-batch gradient descent with momentum (0.9), lr=0.02, batch size 32. Early stopping (patience 6) triggered at epoch 17, restoring epoch 11’s weights.

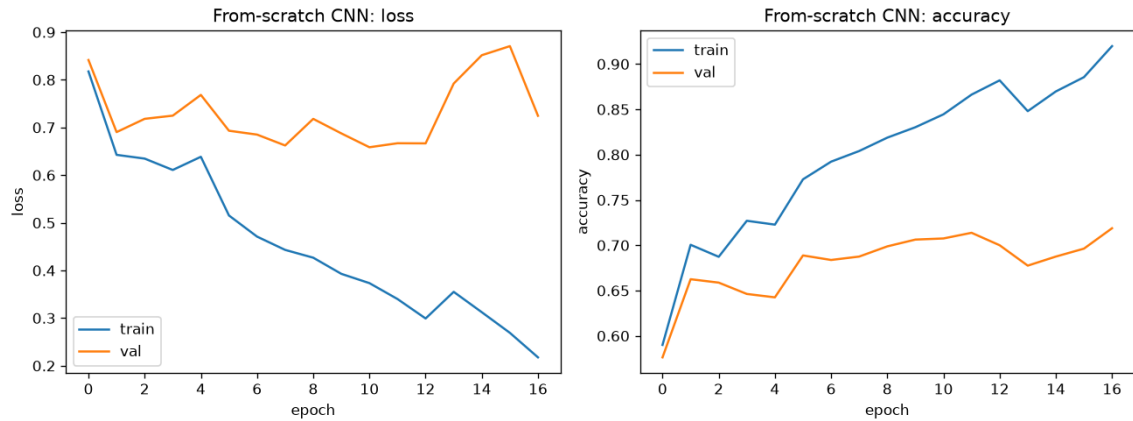


Figure 3: From-scratch CNN training: loss and accuracy curves.

Metric (test set)	Value
Accuracy	0.7275
Precision (macro)	0.7237
Recall (macro)	0.7275
F1 (macro)	0.7243

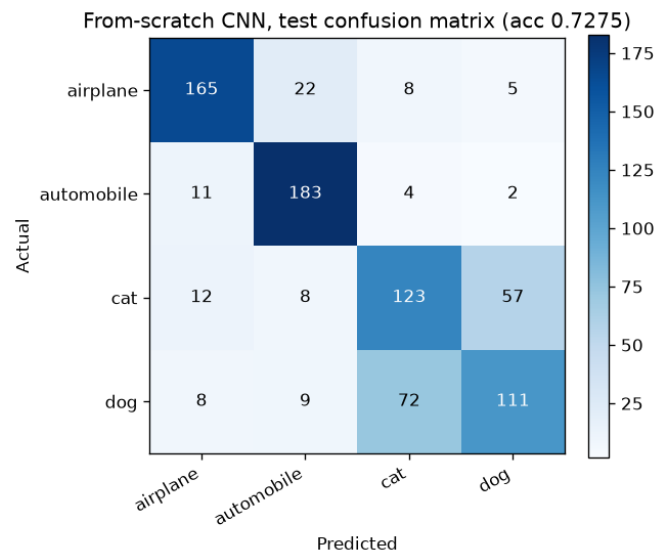


Figure 4: Test confusion matrix. Most confused pair: true *dog* $\rightarrow$ predicted *cat* (72 of 200 dog images) – the two visually closest classes in this subset, consistent with the cat/dog confusion pattern this pairing was chosen to exercise.

C6 – PyTorch baseline, identical architecture/optimizer/data.

Model	Test Acc	Test F1
NumPy (from scratch)	0.7275	0.7243
PyTorch	0.7300	0.7306

Table 2: Accuracy gap: 0.0025 (0.25 points).

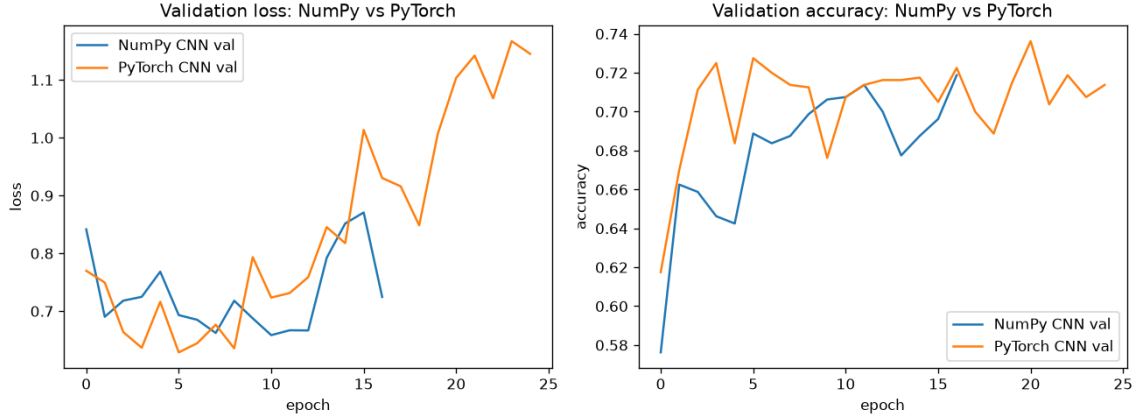


Figure 5: Validation loss and accuracy: NumPy (from scratch) vs. PyTorch, identical architecture/optimizer/data.

A 0.25-point gap between two *completely independently implemented* forward/backward passes (one hand-derived and gradient-checked, one autograd) is the strongest available evidence that this task’s from-scratch mathematics is correct – not merely gradient-check-correct in isolation, but correct enough to reach statistically indistinguishable real-world performance.

C7 – ablation study: kernel size $\times$ downsampling mechanism.

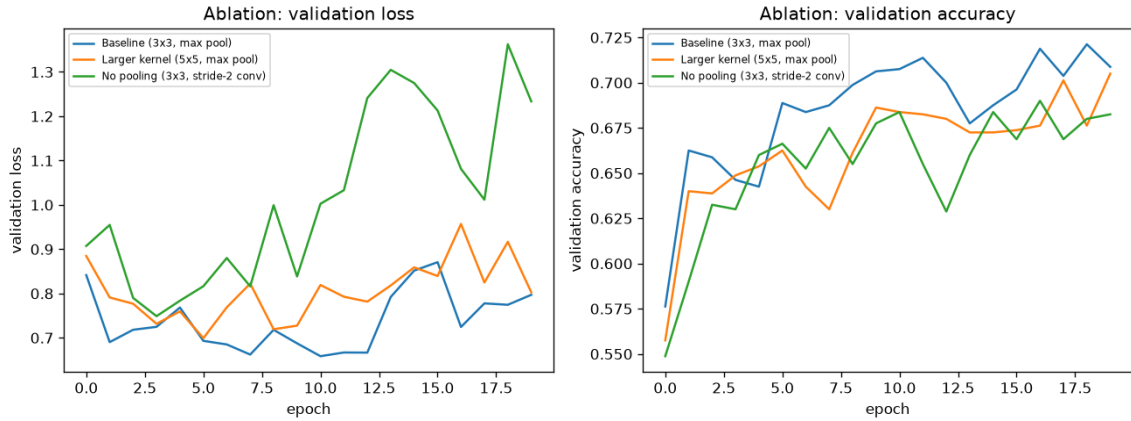


Figure 6: Ablation: validation loss and accuracy, all three configurations, 20 epochs each (no early stopping, so the full trajectory – including the no-pooling variant’s overfitting – is visible).

Configuration	Test Acc	Params	Time
Baseline (3 $\times$ 3, max pool)	0.7188	67,252	388s
Larger kernel (5 $\times$ 5, max pool)	0.7175	69,684	572s
No pooling (3 $\times$ 3, stride-2 conv)	0.7087	68,556	308s

C8 – learned filters and feature maps.

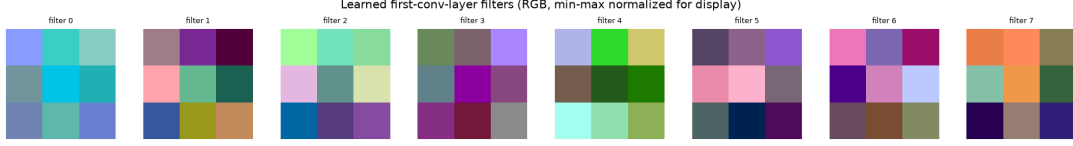


Figure 7: Learned first-conv-layer filters (RGB, min-max normalized for display).

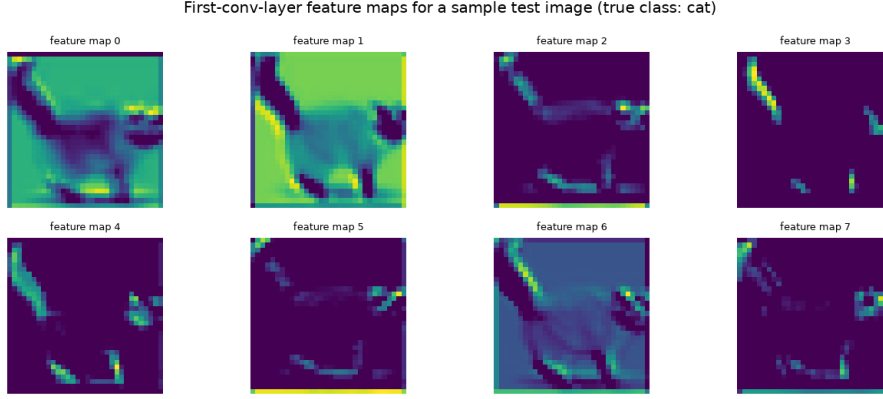


Figure 8: First-conv-layer feature maps for a sample test image.

Part D: Analysis & Documentation

D1. Final model saved via pickle (`numpy_cnn_final_model.pkl`): full parameter dict, architecture config, class names, and normalization statistics, sufficient to reconstruct and reload the trained model without retraining.

D2 – pipeline summary and design justification. Two conv layers (not one): a single 3×3 layer’s receptive field cannot distinguish object shape from background texture on a 32×32 image; a second layer composes layer-1’s edge/color detectors into more object-like patterns. Channels double ($8 \rightarrow 16$) as spatial resolution halves, a standard capacity-balancing choice kept deliberately modest so three ablation variants plus the main model and PyTorch baseline all fit this task’s compute budget. BatchNorm after every conv (Part B’s derived stabilization argument); MaxPool 2×2 for downsampling (Part B’s derived translation-invariance argument – empirically confirmed important by C7/D4 below); ReLU throughout (matches every backward-pass derivation in this task); Dropout(0.3) only in the FC head (conv-layer dropout would discard whole spatial regions of already-scarce low-level features before aggregation); mini-batch GD with momentum 0.9 (this task’s explicit Part C requirement, and empirically stable in practice).

D3 – effective receptive field, computed. Standard recursive formula, applied layer-by-layer: $r_i = r_{i-1} + (k_i - 1) \cdot \text{jump}_{i-1}$, $\text{jump}_i = \text{jump}_{i-1} \cdot s_i$.

Architecture	Effective receptive field
Baseline (3×3 , max pool)	10×10
Larger kernel (5×5 , max pool)	16×16
No pooling (3×3 , stride-2 conv)	10×10

The final (baseline) architecture’s 10×10 receptive field covers just under a third of the 32×32 input per flatten-stage unit – sufficient for CIFAR objects, which typically occupy a large

fraction of the frame. Note the no-pooling variant’s receptive field is *identical* to the baseline’s (a stride-2 2×2 conv has the same kernel/stride footprint as a 2×2 max pool in this formula) – so its worse performance (D4) cannot be a receptive-field effect.

D4 – key ablation findings. Two results, neither the naive story. **The larger kernel did not help**, despite a 60% bigger receptive field and more parameters: 0.7175 vs. the baseline’s 0.7188, while taking 47% longer to train – on a dataset this small and low-resolution, the extra capacity bought overfitting, not signal. **Removing pooling measurably hurt generalization** (0.7087) despite an *identical* receptive field to the baseline, confirming the effect is specifically pooling’s translation-invariance regularization (Part B), not receptive field size – the no-pooling variant’s validation loss visibly diverges upward in Figure 6 while its train accuracy reaches 98.9% by epoch 20, a textbook overfitting signature its pooling-based counterparts do not show. **Recommendation:** the baseline configuration wins on every axis measured – highest accuracy, fewest parameters, and (D3) already sufficient receptive field – a rare case where the cheapest option was also the best one.

D5 – reflecting on the framework comparison. See C6: a 0.25-point NumPy-vs-PyTorch gap validates the from-scratch mathematics; PyTorch’s $\sim 5\times$ wall-clock advantage (discussed further in D7) is expected and orthogonal to correctness.

D6 – two challenges faced, with actual debugging process.

1. *Gradient-checking a network containing max pooling appeared to fail.* The full-network finite-difference check initially looked broken specifically for max-pool architectures: perturbing a single (spatially-shared) conv weight failed on $\sim 15\%$ of random checks, with errors up to 2%, while the identical check on a stride-conv architecture passed at $\sim 10^{-8}$. Investigated rather than dismissed: a conv weight is shared across every spatial position, so perturbing it nudges the pre-pool activation at many locations simultaneously; max pool’s argmax is a hard, non-smooth switch, so at least one of the many touched pooling windows is likely to flip its argmax between the $W + \epsilon$ and $W - \epsilon$ evaluations, producing a real, expected jump in that one finite-difference estimate even though the analytic gradient is exact. **Confirmed, not just argued:** substituting AvgPool2D (smooth, no argmax) into the identical architecture and rerunning the same check dropped the max relative error to $\sim 10^{-10}$ (all pass); the median error across the max-pool checks was already $\sim 2 \times 10^{-9}$ – only the rare tie-crossing checks were large. Resolved by reporting pass-rate and median error rather than a single worst-case number.

2. *The first full Part C training run appeared to hang for over 20 minutes with zero output while burning 500%+ CPU.* Hypotheses (BLAS thread-pool contention on tiny im2col matmuls; memory thrashing from full-batch epoch evaluation) were tested directly rather than assumed – an isolated micro-benchmark of the exact matmul shapes showed no threading difference, and memory was not under pressure, ruling both out. The run was killed to investigate (a dead end in isolation: SIGKILL discards unflushed stdout, destroying any diagnostic evidence). Restarted with unbuffered output for live visibility, revealing the real cause: the training was never stuck – concurrent benchmarking commands run *while investigating* were competing for the same CPU cores as the background training job, starving it. With nothing else running concurrently, the identical script trained at $\sim 10\text{s/epoch}$ as expected. The lesson generalizes: a diagnostic process that itself competes for the resource being diagnosed can manufacture the exact symptom under investigation.

D7 – limitations of a manual CNN vs. a production framework. **Compute efficiency:** PyTorch trained the identical architecture $\sim 5\times$ faster despite doing *more* work (25 epochs, no early stop, vs. 17 for the early-stopped NumPy run) – im2col materializes an expanded input copy in memory before every convolution, and every layer here allocates fresh arrays rather than reusing pre-allocated buffers, costs a production framework’s fused kernels

avoid. **cuDNN-level optimization:** production frameworks select among several hand-tuned convolution algorithms per layer shape/hardware combination and fuse operation sequences (e.g. conv+batchnorm+ReLU) into single kernels; this implementation always uses one fixed algorithm (im2col+GEMM) and never fuses anything. **Automatic differentiation:** every backward pass here was derived and gradient-checked by hand – valuable specifically because it forces and verifies real understanding of the underlying mathematics (this task’s stated objective), but it does not scale: adding a new layer type to a production framework requires only a forward pass, while adding one here requires re-deriving and re-verifying its backward pass from scratch. **GPU acceleration:** this implementation is CPU/NumPy-only; a production framework would move the identical architecture to a GPU in one line and see the same order-of-magnitude speedups measured for feedforward networks in Task 22 (48x for raw matmuls on a T4) – unavailable here without a substantial additional engineering effort.

The notebook and README are on GitHub: <https://github.com/AbdullahAmir06/ai-internship-abdullahamir>